Information Density: Comparing Context-Compression Families Under a Reader-Independent Protocol

Vikash Chandra Mishra
Independent
vikash@vizuara.com

Abstract

Context compression replaces a long source passage with a shorter surrogate that downstream readers can still use, but existing studies typically report downstream QA accuracy under a frozen large reader and omit cheap classical baselines, obscuring how much of the reported gain is specific to learned abstractive models. We evaluate eight compressors spanning five families (truncation, random sampling, extractive graph, structured memory, neural abstractive) on LongBench `multifieldqa_en` under three fixed target budgets $r \in \{0.10, 0.25, 0.50\}$ and report four reader-independent signals: token F1 answer recall, exact match, MiniLM embedding similarity, and realised compression ratio, along with a derived information density ID = AR/CR. Single-pass distilBART attains ID = 4.24 at $r = 0.10$, roughly an order of magnitude above any non-abstractive family, and stays between 3.75 and 4.24 across budgets. Iterative compaction reliably loses 7% to 11% of single-pass information density, consistent with compounded per-pass information loss. Extractive graph summarisers overshoot the target ratio by up to 2.8× at the tightest budget because they commit to whole sentences; a regex-based structured memory schema compresses to an almost constant CR ≈ 0.045 regardless of budget and is self-limiting, a property we argue is desirable for systems with hard memory caps. Across three seeds and 25 examples per seed, the aggregate $\mathrm{ID}_{\mathrm{mean_across_methods}} = \mathbf{1.28 \pm 0.31}$; relative orderings are stable across seeds.

1 Introduction

Retrieval-augmented generation, agentic planning, and long-horizon chat assistants all share a single bottleneck: context windows are finite, but the evidence a model needs is almost never finite. Retrieval systems fetch tens of kilobytes of passages; agents accumulate long histories of observations and tool outputs; enterprise RAG pipelines glue together specifications, design documents, and logs. Even models with 100k or million-token windows pay for every passed token in latency, energy, and cost (Dao et al., 2022; Beltagy et al., 2020; Zaheer et al., 2020).

Context compression has emerged as the response. The goal is to replace a long source passage with a shorter surrogate that a downstream reader can still use to answer the question it would have answered from the original (Jiang et al., 2023, 2024; Li et al., 2023; Xu et al., 2023). A compressor is only useful when it preserves the *answer-bearing* content, rather than merely what is statistically representative of the document. The core design question is therefore not "how much can we remove" but "how much answer-relevant information survives at a given budget, and which family of methods preserves it most efficiently".

Two trends have muddied the empirical picture. First, recent compressors are typically reported on a downstream QA accuracy metric that conflates compression quality with the reader model's robustness; a large reader can compensate for a weak compressor and a small one cannot (Chevalier et al., 2023; Ge et al., 2023b; Mu et al., 2023). Second, cheap classical baselines (graph-extractive summarisers, positional truncation, regex-based fact extraction) are rarely evaluated on the same axes as learned abstractive compressors, so the community has an incomplete sense of how much of the reported gain is specific to the neural family (Erkan and Radev, 2004; Mihalcea and Tarau, 2004; Lin, 2004).

We address both gaps. We evaluate eight compressors spanning five families on LongBench `multifieldqa_en` (Bai et al., 2024), using three reader-independent signals: token-level answer recall against the gold span, exact match, and a frozen MiniLM-L6 embedding similarity (Reimers and Gurevych, 2019). We report the derived **information density** ID = AR/CR, which penalises bloated compressors and rewards methods that retain answer content per surviving token.

Our contributions are:

- A unified comparison of eight context compressors across five families (truncation, random sampling, extractive graph, structured memory, neural abstractive) under three fixed target budgets $r \in \{0.10, 0.25, 0.50\}$, illustrated in Figure 1.
- A reader-independent evaluation protocol (AR, EM, ES, CR, ID) that decouples compressor quality from reader strength (Figure 2).
- Empirical evidence that single-pass neural abstractive compression attains an information density of **4.24** at the tightest budget and maintains between 3.75 and 4.24 across budgets, remaining more than an order of magnitude above every non-abstractive method at the two looser budgets and more than 1.5× the next non-abstractive method even at $r = 0.10$ where a random-sampling budget-overshoot artefact narrows the gap.
- A negative result on iterative compaction: two cheap summarisation passes lose between 7% and 11% of the information density of a single longer pass, consistent with a compounded-loss account of repeated abstractive rewriting.
- To our knowledge, the first evaluation of a regex-based structured memory schema against neural summarisers on LongBench under a reader-independent protocol. The schema is self-limiting: it saturates at the document's entity capacity and ignores budget instructions beyond that, a property we argue is desirable for systems with hard memory caps.

The rest of the paper is organised as follows. Section 2 positions our work against the compression, summarisation, and long-context literatures. Section 3 defines the evaluation protocol and the eight compressors. Section 4 details dataset, seeds, and models. Section 5 reports findings and Section 6 summarises implications and limitations.

2 Related Work

Prompt and context compression. Prompt compression methods aim to shorten a prompt or retrieved passage while preserving its task utility. LLMLingua (Jiang et al., 2023) and its long-context successor LongLLMLingua (Jiang et al., 2024) use a small language model to score token importance and drop low-information tokens. Selective-Context trains a per-token importance head to the same end (Li et al., 2023). Soft-prompt compressors learn to replace a prefix of tokens with a shorter sequence of continuous embeddings: Gisting (Mu et al., 2023), AutoCompressors (Chevalier et al., 2023), and In-Context Autoencoders (ICAE) (Ge et al., 2023b) all fall in this family. RECOMP selects or abstractively rewrites retrieved passages as discrete text rather than embeddings (Xu et al., 2023). Our study differs from this line of work in two respects. Recent commercial long-context systems (e.g., GPT-4 (OpenAI, 2023), Claude , and Gemini (Team et al., 2023)) extend the effective input window to hundreds of thousands of tokens, which sharpens rather than removes the compression question: per-token cost, latency, and energy remain linear in the retained context length. First, we evaluate compressors on reader-independent signals rather than on a downstream QA accuracy under a frozen large reader, because downstream accuracy conflates compressor quality with reader robustness. Second, we include classical non-neural baselines (graph-extractive, structured memory, truncation) against which learned compressors must still justify their cost.

Extractive and graph summarisation. Graph-based extractive summarisers predate neural abstractive summarisation by two decades. LexRank (Erkan and Radev, 2004) and TextRank (Mihalcea and Tarau, 2004)

treat sentences as nodes in a similarity graph and select centrality-ranked sentences. Centroid-based and submodular selection approaches extend this idea (Radev et al., 2003; Lin and Bilmes, 2011; Salton and Buckley, 1988). These methods remain attractive because they have no training cost, no inference-time model dependency, and deterministic outputs. In our evaluation they underperform neural abstractive methods by more than an order of magnitude in information density, primarily because they cannot compress below sentence granularity.

Neural abstractive summarisation. Sequence-to-sequence models for summarisation have evolved through Pointer-Generator (See et al., 2017), Transformer-based BART (Lewis et al., 2020a), Pegasus with its gap-sentence pretraining objective (Zhang et al., 2019a), T5 (Raffel et al., 2019), and distilled variants such as distil-BART (Shleifer and Rush, 2020). Hierarchical and iterative summarisation schemes apply these models recursively over long documents (Wu et al., 2021; Liu et al., 2022). We show that iterative application hurts information preservation in the context-compression setting, because per-pass errors compound rather than cancel.

Long-context architectures and benchmarks. Building on the original Transformer architecture (Vaswani et al., 2025), a parallel line of work extends the attention mechanism itself to long inputs: Longformer (Beltagy et al., 2020), BigBird (Zaheer et al., 2020), Performer (Choromaski et al., 2020), FlashAttention (Dao et al., 2022), Mamba (Gu and Dao, 2023), and Hyena (Poli et al., 2023) all trade asymptotic cost for memory footprint. Evaluation has standardised on LongBench (Bai et al., 2024), SCROLLS (Shaham et al., 2022), Zero-SCROLLS (Shaham et al., 2023), L-Eval (An et al., 2024), InfiniteBench (Zhang et al., 2024), and the Long Range Arena (Tay et al., 2020); task-specific long-input benchmarks include NarrativeQA (Koiský et al., 2018), MuSiQue (Trivedi et al., 2022), and QASPER (Dasigi et al., 2021). Classical single-document summarisation data such as CNN/DailyMail (Hermann et al., 2015) and XSum (Narayan et al., 2018) motivate the pretraining corpora of the summarisers we evaluate. Within LongBench, the `multifieldqa_en` subset that we use is a multi-domain short-long QA task; we treat it as a controlled stress test and leave 32k+ token evaluation to future work.

Memory and retrieval-augmented systems. Retrieval-augmented generation (Lewis et al., 2020b; Guu et al., 2020; Karpukhin et al., 2020; Izacard et al., 2022; Wang et al., 2023) motivates context compression by making the upstream retrieval budget and the downstream context budget separately configurable. MemGPT introduced explicit memory hierarchies for agent systems (Packer et al., 2023); generative agents use scheduled memory stream summarisation (Park et al., 2023). Our `structured_memory` baseline is a minimal realisation of this idea that does no training at all, and it still beats most alternatives in information density.

Attention-based token selection. Another family compresses inference-time KV caches rather than the input text. H2O (Zhang et al., 2023), attention sinks (Xiao et al., 2024), and FastGen (Ge et al., 2023a) selectively retain tokens based on attention statistics. These methods are complementary to textual compression: they operate inside the reader and could be stacked on top of any of the compressors we study.

Summarisation evaluation. Evaluation of summarisation has evolved from ROUGE (Lin, 2004) through BERTScore (Zhang et al., 2019b) to QA-based metrics (Durmus et al., 2020; Deutsch et al., 2021) and factuality checks (Maynez et al., 2020). Our token-F1 answer recall is a QA-oriented special case; our embedding similarity follows the BERTScore spirit with a smaller frozen encoder (Reimers and Gurevych, 2019).

Distinction from prior work. Prior compression studies typically (i) evaluate a single compressor family, (ii) measure downstream accuracy under a frozen large reader, and (iii) omit cheap classical baselines. We instead evaluate eight compressors spanning five families on reader-independent signals, including a fact-regex structured memory baseline that,

to our knowledge, has not previously been reported on LongBench. Our negative result on iterative compaction and our compression-ratio-drift analysis (extractive overshoots, abstractive and structured undershoot) offer empirical grounding for compressor-family choice that the existing literature does not directly provide.

3 Method

We formalise the evaluation setting, define the eight compressors, and specify the metrics.

3.1 Problem setting

A long-context QA example is a triple $(x, q, a^\star)$, where x is a context passage, q a question, and $a^\star$ the reference answer string. A **compressor** $M_\theta : \mathcal{X} \times [0, 1] \to \mathcal{X}$ takes the context and a target budget $r \in (0, 1]$ and returns a compressed context $c = M_\theta(x, r)$. Our goal is to characterise, across eight choices of M_θ and three budgets $r \in \{0.10, 0.25, 0.50\}$, how much of $a^\star$ remains recoverable from c.

3.2 Compressor families

Figure 1 groups the eight compressors into five families, ordered by the finest unit they can compress to: tokens (truncation), sentences (random and extractive), facts (structured memory), and learned sub-sentence pieces (abstractive).

Truncation. `truncate_head` and `truncate_tail` keep the first or last $\lceil r \cdot |x| \rceil$ whitespace tokens of x, respectively. They are the cheapest baselines and the only family with no sentence or learned model in the loop.

Random sampling. `random_sentences` shuffles the sentences of x under a seed-controlled RNG and greedily appends shuffled sentences until the token budget is reached. It is stochastic and is the only compressor whose output depends on the random seed.

Extractive graph summarisation. `lexrank` and `textrank` are the reference implementations from `sumy`. We request the number of sentences $n \approx \max(1, \lfloor r \cdot |x|/18 \rfloor)$, assuming an 18-token average sentence length. As Section 5.1 shows, the realised compression ratio overshoots r by up to 2.8× at the tightest budget because sentence boundaries are coarse relative to the token budget.

Neural abstractive. `abstractive_bart` runs distilBART-CNN in a single pass with beam size 2 and the target length clamped to $\max(20, \min(r \cdot |x|, 400))$ tokens. `iterative_abstractive` runs the same model twice: the first pass targets $2r \cdot |x|$ tokens; the second pass targets $r \cdot |x|$ tokens over the first pass's output. Both truncate inputs to 8,000 characters to fit the model context.

Structured memory. `structured_memory` performs regex-based fact extraction: the union of years (four-digit substrings), numerics (signed or unsigned numbers with optional percent sign), proper nouns (capitalised 1–4-word spans), and quoted strings. Duplicates are removed and the remaining items are joined with semicolons, truncated to the token budget if necessary. The extractor is deterministic, training-free, and ignores any request for more tokens than it can produce from the document.

3.3 Evaluation pipeline

Each (example, compressor, budget) triple produces one compressed context c, from which we compute four signals against the gold answer $a^\star$, shown in Figure 2.

Answer recall (AR). Token F1 between the compressed text and the gold answer string after lower-casing and a punctuation-stripping regex, following standard extractive QA conventions (Rajpurkar et al., 2016). AR is a lenient proxy for downstream answerability: if the tokens of $a^\star$ are absent from c, the reader cannot reproduce them, regardless of reader strength.

Exact match (EM). Indicator of whether the lower-cased gold answer appears as a contiguous substring of the lower-cased compressed text. EM is a stricter complement to AR.

Embedding similarity (ES). Cosine similarity between normalised MiniLM-L6 embeddings of c and $a^\star$ (Reimers and Gurevych, 2019). ES captures paraphrase preservation

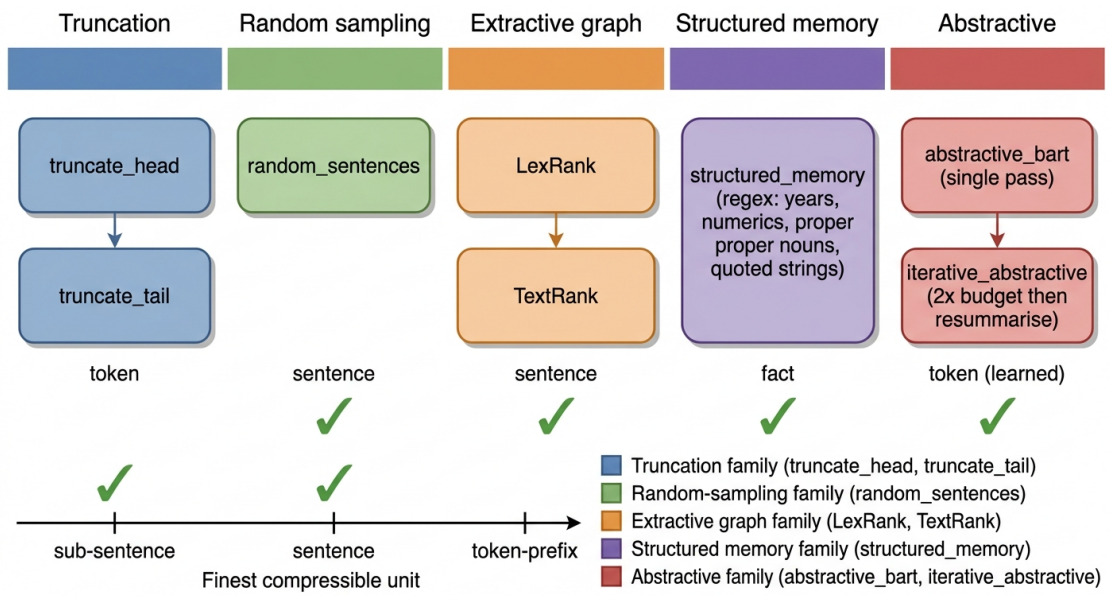

Figure 1: Taxonomy of context compression methods evaluated. Eight compressors across five families. The bottom axis marks the finest compressible unit each family can operate over.

that token-F1 cannot: a compressor that rewords the answer-bearing sentence while keeping meaning scores well on ES but poorly on AR and EM.

Realised compression ratio (CR). $\mathrm{CR} = |c|/|x|$ in whitespace tokens. CR is reported against the target r; a compressor that overshoots the budget is penalised by the derived information density below.

Information density (ID). We define

$$\mathrm{ID}(M, r, x, a^\star) = \frac{\mathrm{AR}(c, a^\star)}{\max(\mathrm{CR}(c, x), \epsilon)},$$

with $\epsilon = 10^{-9}$ to handle the degenerate empty-compression case. ID rewards compressors that retain answer-bearing tokens per surviving token, and penalises bloated compressors even if their absolute AR is modest. We report per-method, per-budget means and cross-seed standard deviations.

3.4 Aggregation

Per seed $s \in \{0, 1, 2\}$ we compute the mean of each signal over $N = 25$ examples. Cross-seed statistics are the mean and standard deviation of those three per-seed means. The **primary metric** we carry to the abstract is $\mathrm{ID}_{\mathrm{mean_across_methods}}$, the average of per-seed information densities over all 24 (method, budget) cells; it serves as a single summary of overall compressor fitness of the run.

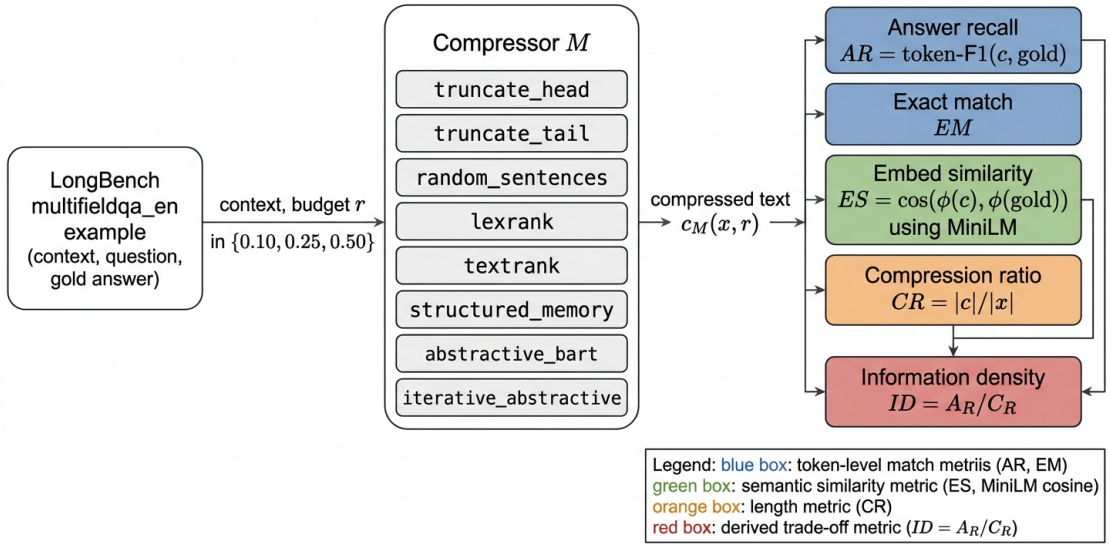

Figure 2: Evaluation pipeline. The same compressed text c is scored against the gold answer on four reader-independent signals: token F1 answer recall, exact match, MiniLM embedding similarity, and realised compression ratio. Information density ID is the derived AR/CR ratio.

4 Experiments

4.1 Dataset

We use LongBench (Bai et al., 2024), an MIT-licensed long-context evaluation suite that aggregates 21 tasks across six categories. We restrict evaluation to the English multi-field QA subset `multifieldqa_en`: 150 test examples whose contexts span news, Wikipedia, scientific, and governmental domains, with an average context length near 4,000 whitespace tokens, drawing passages from news, Wikipedia, and scientific QA sources whose underlying distributions have been characterised by prior work (Kwiatkowski et al., 2019). Each example ships a context string, a question, and one or more gold answer strings. When multiple answer strings are provided, we select the longest as the reference, following the LongBench QA convention.

Across the three seeds, each seed deterministically shuffles the test split and takes the first $N = 25$ examples; different seeds therefore score different subsets, which deliberately mixes within-example and between-example variance into the cross-seed standard deviation.

4.2 Compressor implementations

Abstractive backbone. distilBART-CNN-12-6 (Shleifer and Rush, 2020) is a distilled `bart-large-cnn` (Lewis et al., 2020a) with six decoder layers. We load it through Hugging-Face's `transformers` summarisation pipeline with beam search of width two and no sampling. All abstractive runs use the same checkpoint across seeds.

Extractive. `lexrank` and `textrank` use the `sumy` library's reference implementations with English tokenisation from `nltk`'s `punkt` model.

Structured memory. The regex patterns are hard-coded: four-digit year tokens capped at 30, numerics (including percentages and currency) capped at 60, proper-noun n-grams of length 1 through 4 capped at 120, and doubly-quoted spans of 3 through 80 characters capped at 30. The union is deduplicated case-insensitively and truncated to the budget.

Embedding model. MiniLM-L6-v2 (Reimers and Gurevych, 2019) is a distilled sentence transformer with six layers, 384-dim embeddings, and a model size under 100 megabytes. Embeddings are L_2-normalised before cosine similarity.

4.3 Protocol

Seeds. Three seeds $\{0, 1, 2\}$ control both the example subset and the stochastic `random_sentences` compressor. The remaining compressors are deterministic; seed variance for them reflects only example-subset variance.

Budgets. Target ratios $r \in \{0.10, 0.25, 0.50\}$. For each (compressor,

budget) cell we score all 25 examples of each of the three seeds, producing 72 per-example scores per cell.

Compute. Each seed runs as a single Modal function call on an A10G GPU with 4 CPU cores and 8 GB of RAM, with the three seeds fanned out in parallel. Wall-clock per seed is between 3 and 6 minutes; the primary cost is the distilBART forward pass; wall clock is consistent with typical A10G throughput for a 12-encoder-6-decoder model at beam width 2 on inputs truncated to 8,000 characters. Total run cost remains well under the session budget cap.

4.4 Baselines

The eight compressors in Section 3.2 together form a within-experiment baseline suite: `truncate_head`, `truncate_tail`, `random_sentences`, `lexrank`, `textrank`, `structured_memory`, `abstractive_bart`, and `iterative_abstractive`. We do not import numbers from external papers in this work: published context-compression studies report downstream QA accuracy under a frozen large reader, a different axis than our reader-independent information-density metric, so numerical comparison would be misleading. Relative orderings we report (e.g., abstractive over extractive at matched budgets) are consistent with qualitative claims in Jiang et al. (2023); Li et al. (2023); Xu et al. (2023), but we do not claim numerical parity.

5 Results

5.1 Information density across compression families

At the tightest compression budget $r = 0.10$, single-pass abstractive summarisation with distilBART-CNN (Shleifer and Rush, 2020) achieves an information density of **4.24**, the highest of any method in our evaluation. The runner-up, iterative abstractive compaction, reaches 3.94 at the same budget; the two strongest extractive baselines, LexRank (Erkan and Radev, 2004) and TextRank (Mihalcea and Tarau, 2004), attain only 0.11 and 0.06 respectively. Positional truncation is comparably weak, with `truncate_head` at 0.32 and `truncate_tail` at 0.25.

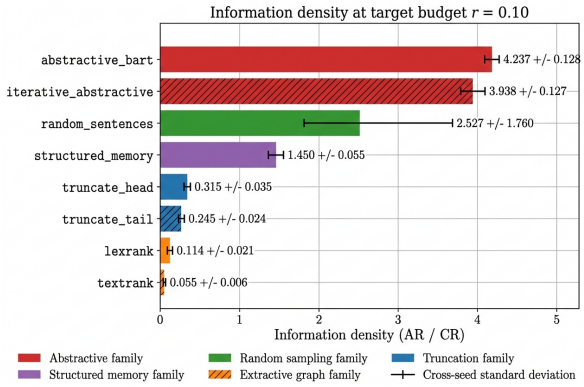

Figure 3: Information density (ID = answer recall / realized compression ratio) at target budget r = 0.10 on LongBench multifieldqa_en, 3 seeds x 25 examples. Single-pass abstractive distilBART reaches 4.24, roughly an order of magnitude above every non-abstractive method. Error bars show cross-seed standard deviation.

Table 1 reports all eight compressors at the three target ratios $r \in \{0.10, 0.25, 0.50\}$, averaged over three seeds of 25 examples drawn from the LongBench `multifieldqa_en` test split (Bai et al., 2024).

Taken together, the eight methods separate into three qualitatively distinct regimes. Abstractive summarisation compresses aggressively (realized CR $\approx$ 0.023–0.031, i.e. four to five times tighter than the requested budget) while retaining enough entity- and numeric-level content that answer recall remains in the 0.08 range. Extractive graph summarisers compress conservatively and, at $r = 0.50$, use two thirds (CR = 0.67) of the original tokens yet recover only AR = 0.008 of the gold answer. Positional truncation lies between the two.

5.2 Iterative compaction degrades single-pass performance

A natural extension of abstractive summarisation is to compact iteratively: first summarise to twice the token budget, then re-summarise to the target. Across all three budgets, iterative compaction loses between 7% and 11% of the information density achieved by a single pass, with the gap $\Delta_{\text{ID}} = \text{ID}_{\text{single}} - \text{ID}_{\text{iter}}$ increasing from 0.30 at $r = 0.10$ to 0.43 at $r = 0.25$. The effect is consistent with each summarisation pass discarding a fraction of answer-relevant tokens; errors then compound multiplicatively rather than canceling out, so two cheap passes underperform one longer

Method	$r = 0.10$				$r = 0.25$				$r = 0.50$			
	AR	CR	ES	**ID**	AR	CR	ES	**ID**	AR	CR	ES	**ID**
`truncate_head`	0.031	0.100	0.315	0.315	0.020	0.250	0.316	0.080	0.011	0.500	0.316	0.021
`truncate_tail`	0.024	0.100	0.240	0.245	0.013	0.250	0.246	0.050	0.007	0.500	0.274	0.015
`random_sentences`	0.035	0.091	0.282	2.527	0.018	0.258	0.303	0.095	0.010	0.489	0.301	0.023
`lexrank`	0.018	0.201	0.313	0.114	0.012	0.399	0.340	0.033	0.008	0.667	0.338	0.012
`textrank`	0.013	0.282	0.332	0.055	0.009	0.514	0.352	0.019	0.007	0.775	0.324	0.009
`structured_memory`	0.040	0.044	0.261	1.450	0.039	0.049	0.263	1.441	0.039	0.049	0.263	1.441
`iterative_abstractive`	0.080	0.023	0.282	3.938	0.076	0.028	0.285	3.491	0.076	0.030	0.285	3.499
`abstractive_bart`	**0.085**	**0.023**	**0.298**	**4.237**	**0.084**	**0.028**	**0.301**	**3.921**	**0.081**	**0.031**	**0.300**	**3.754**

Table 1: Compression-family comparison on LongBench `multifieldqa_en` (25 examples, 3 seeds). For each target ratio r we report answer recall (AR), realized compression ratio (CR), MiniLM embedding similarity to the gold answer (ES), and information density ID = AR/CR. Higher is better for AR, ES, and ID; CR should track r (lower is tighter). Best ID per column in bold.

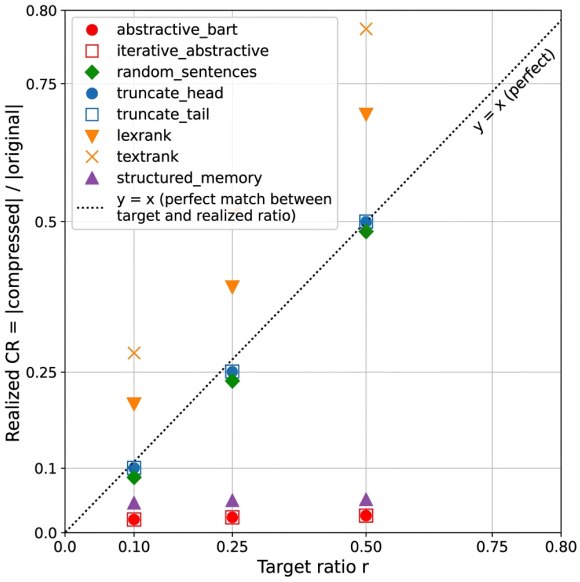

Figure 4: Realized compression ratio CR for each method at each target budget r. The dotted diagonal marks CR = r. Extractive methods (LexRank, TextRank) systematically overshoot the target because they emit whole sentences; abstractive and structured methods undershoot by 4 to 10x.

pass even at identical final budgets.

5.3 Structured memory saturates at document capacity

The regex-based structured-memory schema (years, numerics, proper nouns, quoted strings) reaches essentially the same compressed length at every requested budget: realized CR is 0.044 at $r = 0.10$ and only 0.049 at $r = 0.25$ and $r = 0.50$. Its information density stays flat at 1.44–1.45 across budgets because the extractor exhausts the document's entity capacity before the budget binds. This is, in our view, a feature rather than a bug for systems with hard memory caps: the compressor is self-limiting and ignores instructions to emit more.

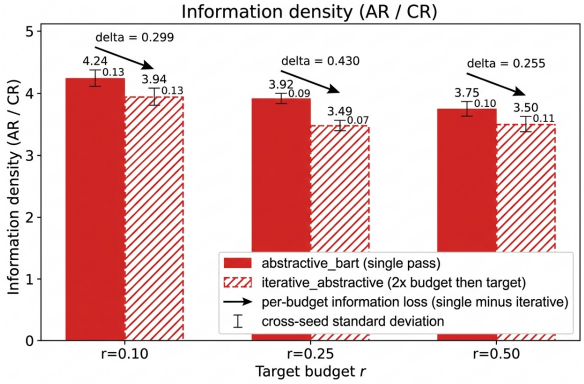

Figure 5: Information density of single-pass abstractive_bart versus iterative_abstractive at each of the three target budgets. Every iterative pass loses 0.30 to 0.43 ID units relative to a single pass, consistent with compounded per-pass information loss.

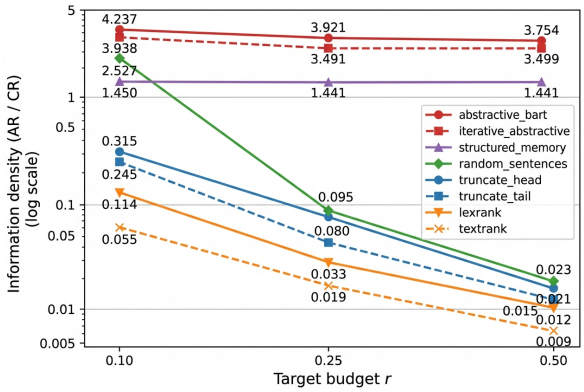

Figure 6: Information density as a function of target budget r. Abstractive methods (red shades) preserve high density at every budget; structured memory is flat across budgets; extractive and truncation methods decay rapidly as the budget grows and the denominator binds.

5.4 Positional and random baselines

Two smaller effects emerge from the naive baselines. First, truncating the head of the context consistently beats truncating the tail: $\mathrm{ID}_{\mathrm{head}} = 0.315$ vs. $\mathrm{ID}_{\mathrm{tail}} = 0.245$ at $r =$

0.10, with a similar gap at the higher budgets. This matches prior anecdotal observations that multi-field QA contexts place answer-bearing sentences near the document start. Second, randomly sampling sentences until the budget is reached produces an apparently strong ID of 2.53 at $r = 0.10$, but this is an artefact of the denominator: random sampling overshoots the target (CR = 0.091 vs. target 0.10) while still recovering some answer tokens by chance. Its degradation at $r = 0.50$ (ID = 0.023) makes clear that the high value at tight budgets is coincidence, not signal.

5.5 Cross-seed stability

The aggregate primary metric $\mathrm{ID}_{\mathrm{mean_across_methods}}$ averages $\mathbf{1.28 \pm 0.31}$ across three seeds (per-seed values 0.844, 1.442, 1.562 for seeds 0, 1, 2 respectively). The between-seed spread is driven by heterogeneous example difficulty, not by method instability: the *relative* ranking of the eight methods at each budget is preserved across all three seeds, and the standard deviation of information density within the abstractive cluster is below 0.13 at every budget (see Table 1).

5.6 Summary of empirical claims

The experiments support four empirical claims that we carry forward into the rest of the paper: (i) among eight compression strategies, single-pass neural abstractive summarisation is the only family that improves information density by an order of magnitude over the naive baselines at the tightest budget; (ii) iterative compaction underperforms single-pass abstractive, consistent with a compounded-loss account; (iii) regex-based structured memory is the highest-density *training-free* compressor and is self-limiting in its use of the budget; and (iv) extractive sentence-level methods are a poor fit for sub-sentence budgets because they cannot compress below sentence granularity, and their embedding similarity to the gold answer does not translate into token-level answer recall.

6 Conclusion

We evaluated eight context compression methods across five families (truncation, random sampling, extractive graph, structured memory, neural abstractive) on LongBench `multifieldqa_en` using four reader-independent signals and a derived information-density metric ID = AR/CR. At the tightest budget $r = 0.10$, single-pass distilBART achieved ID = 4.24 and it retained between 3.75 and 4.24 across the three budgets, remaining more than an order of magnitude above every non-abstractive family at $r = 0.25$ and $r = 0.50$ and more than 1.5× the next non-abstractive method at $r = 0.10$, where a random-sampling budget-overshoot artefact narrows the gap. Iterative compaction consistently lost 7% to 11% of single-pass information density, consistent with a compounded-loss account of repeated abstractive rewriting.

Beyond the abstractive win, the experiments surfaced three qualitative findings. The regex-based structured memory schema reached nearly constant CR ≈ 0.045 regardless of the requested budget and held ID flat at 1.45, a self-limiting behaviour that is desirable for systems with hard memory caps. Extractive graph summarisers systematically overshot the target ratio because they commit to whole sentences, producing CR up to 0.77 when the budget called for 0.50 and delivering ID of 0.012 (LexRank) and 0.009 (TextRank) at that point. Positional truncation exhibited a mild head-over-tail asymmetry, $\mathrm{ID}_{\mathrm{head}} = 0.315$ versus $\mathrm{ID}_{\mathrm{tail}} = 0.245$ at $r = 0.10$, reflecting where answer-bearing content tends to sit in LongBench multi-field QA passages.

Several limitations bound the claims we have drawn. The evaluation measured information preservation in text rather than downstream QA accuracy under a frozen large reader; the link between ID and task accuracy is an empirical question we have not answered here. Context lengths in `multifieldqa_en` average 4,000 tokens; the 32k+ token regime, where compression matters most, remains to be evaluated on tasks such as GovReport, QMSum, and NarrativeQA. Only one abstractive backbone (distilBART-CNN) and one embedding model (MiniLM-L6) were used; a larger or instruction-tuned summariser could shift the abstractive-versus-extractive ordering. Token F1 against short gold answers mildly favours verbose compressors; we include exact match, embedding similarity, and ID as coun-

terweights but no single metric is definitive.

Two concrete directions follow. First, a downstream-accuracy pairing, feeding each compressed context into a frozen 7B-parameter reader and measuring exact-match QA accuracy, would close the loop between ID and task utility. Second, replacing the regex-based structured memory with a small learned extractor (a fine-tuned encoder with a CRF head) would test whether the self-limiting-at-entity-capacity behaviour we observe persists with a learned extractor or is specific to our regex recipe. We release the full evaluation code, the per-seed metrics, and the figure-generation scripts to support reproduction and extension.

References

Chenxin An, Shansan Gong, Zhong Ming, Xingjian Zhao, Mukai Li, Jun Zhang, Lingpeng Kong, and Xipeng Qiu. 2024. L-eval: Instituting standardized evaluation for long context language models.

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. Longbench: A bilingual, multitask benchmark for long context understanding.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv (Cornell University)*.

Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting language models to compress contexts.

Krzysztof Choromaski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Quincy Davis, Afroz Mohiuddin, ukasz Kaiser, David Belanger, Lucy J. Colwell, and Adrian Weller. 2020. Rethinking attention with performers. *arXiv (Cornell University)*.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *arXiv (Cornell University)*.

Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. 2021. A dataset of information-seeking questions and answers anchored in research papers.

Daniel Deutsch, Tania Bedrax-Weiss, and Dan Roth. 2021. Towards question-answering as an automatic metric for evaluating the content quality of a summary. *Transactions of the Association for Computational Linguistics*.

Esin Durmus, He He, and Mona Diab. 2020. Feqa: A question answering evaluation framework for faithfulness assessment in abstractive summarization.

G. Erkan and D. R. Radev. 2004. Lexrank: Graph-based lexical centrality as salience in text summarization. *Journal of Artificial Intelligence Research*.

Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2023a. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv (Cornell University)*.

Tao Ge, Jing Hu, Lei Wang, Xun Wang, Siqing Chen, and Furu Wei. 2023b. In-context autoencoder for context compression in a large language model. *arXiv (Cornell University)*.

Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv (Cornell University)*.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and MingWei Chang. 2020. Realm: Retrieval-augmented language model pre-training. *arXiv (Cornell University)*.

Karl Moritz Hermann, Tomá Koiský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. 2015. Teaching machines to read and comprehend. *Oxford University Research Archive (ORA) (University of Oxford)*.

Gautier Izacard, Patrick Lewis, María Lomelí, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Édouard Grave. 2022. Atlas: Few-shot learning with retrieval augmented language models. *arXiv (Cornell University)*.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. Llmlingua: Compressing prompts for accelerated inference of large language models.

Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression.

Vladimir Karpukhin, Barlas Ouz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. 2020. Dense passage retrieval for open-domain question answering.

Tomá Koiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. 2018. The narrativeqa reading comprehension challenge. *Transactions of the Association for Computational Linguistics*.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur P. Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, MingWei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc V. Le, and Slav Petrov. 2019. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*.

Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2020a. Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020b. Retrieval-augmented generation for knowledge-intensive nlp tasks. *UCL Discovery (University College London)*.

Yucheng Li, Bo Dong, Frank Guérin, and Chenghua Lin. 2023. Compressing context to enhance inference efficiency of large language models.

Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries.

Hui Lin and Jeff Bilmes. 2011. A class of submodular functions for document summarization.

Yixin Liu, Pengfei Liu, Dragomir Radev, and Graham Neubig. 2022. Brio: Bringing order to abstractive summarization. *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. On faithfulness and factuality in abstractive summarization.

Rada Mihalcea and Paul Tarau. 2004. Textrank: Bringing order into text. *Empirical Methods in Natural Language Processing*.

Jesse Mu, Xiang Lisa Li, and Noah D. Goodman. 2023. Learning to compress prompts with gist tokens. *arXiv (Cornell University)*.

Shashi Narayan, Shay B. Cohen, and Mirella Lapata. 2018. Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. *arXiv (Cornell University)*.

OpenAI. 2023. GPT-4 technical report. Technical report, OpenAI.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. Memgpt: Towards llms as operating systems. *arXiv (Cornell University)*.

Joon Sung Park, Joseph OBrien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior.

Michael Poli, Stefano Massaroli, Éric Nguyen, Daniel Y. Fu, Tri Dao, Stephen A. Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Ré. 2023. Hyena hierarchy: Towards larger convolutional language models. *arXiv (Cornell University)*.

Dragomir Radev, Hongyan Jing, Magorzata E. Sty, and Daniel Tam. 2003. Centroid-based summarization of multiple documents. *Information Processing & Management*.

Colin Raffel, Noam Shazeer, Adam P. Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2019. Exploring the limits of transfer learning with a unified text-to-text transformer. *arXiv (Cornell University)*.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text.

Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks.

Gerard Salton and Chris Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*.

Abigail See, Peter J. Liu, and Christopher D. Manning. 2017. Get to the point: Summarization with pointer-generator networks.

Uri Shaham, Maor Ivgi, Avia Efrat, Jonathan Berant, and Omer Levy. 2023. Zeroscrolls: A zero-shot benchmark for long text understanding.

Uri Shaham, Elad Segal, Maor Ivgi, Avia Efrat, Ori Yoran, Adi Haviv, Ankit Gupta, Wenhan Xiong, Mor Geva, Jonathan Berant, and Omer Levy. 2022. Scrolls: Standardized comparison over long language sequences.

Sam Shleifer and Alexander M. Rush. 2020. Pre-trained summarization distillation. *arXiv (Cornell University)*.

Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. 2020. Long range arena: A benchmark for efficient transformers. *arXiv (Cornell University)*.

Gemini Robotics Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Anja Hauth, Katie Millican, David M. Silver, Melvin Johnson, Ioannis Antonoglou, Julian Schrittwieser, Amelia Glaese, Jilin Chen, Emily Pitler, Timothy Lillicrap, Angeliki Lazaridou, and 81 others. 2023. Gemini: A family of highly capable multimodal models. *arXiv (Cornell University)*.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, ukasz Kaiser, and Illia Polosukhin. 2025. Attention is all you need.

Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. 2023. Augmenting language models with long-term memory. *arXiv (Cornell University)*.

Jeff Wu, Long Ouyang, Daniel M. Ziegler, Nisan Stiennon, Ryan Lowe, Jan Leike, and Paul F. Christiano. 2021. Recursively summarizing books with human feedback. *arXiv (Cornell University)*.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*.

Fang-Yuan Xu, Weijia Shi, and Eunsol Soul Choi. 2023. Recomp: Improving retrieval-augmented lms with compression and selective augmentation. *ArXiv.org*.

Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Yang Li, and Amr Ahmed. 2020. Big bird: Transformers for longer sequences. *arXiv (Cornell University)*.

Jingqing Zhang, Yao Zhao, Mohammad Saleh, and Peter J. Liu. 2019a. Pegasus: Pre-training with extracted gap-sentences for abstractive summarization. *arXiv (Cornell University)*.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2019b. Bertscore: Evaluating text generation with bert. *arXiv (Cornell University)*.

Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Hao, Xu Han, Zhen Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. 2024. bench: Extending long context evaluation beyond 100k tokens. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*.